

System Modeling

- **Definition:** System modeling is creating **abstract representations** of a system to understand, communicate, and validate its functionality.
- **Purpose:**
 - Helps analysts identify and validate system requirements.
 - Helps engineers understand what to implement.
 - Helps communicate clearly with customers.
- **When used:** Mainly during **requirement engineering** and design.
- **Types of system models:**
 1. **Contextual model** – shows the system's environment and boundaries.
 2. **Interaction model** – shows how users and systems interact.
 3. **Structural model** – shows system components and their relationships.
 4. **Behavioral model** – shows how the system behaves under different conditions.

1. Context Models

- **Definition:** Context models represent the **operational environment of a system**—what is outside its boundaries.
- **Purpose:**
 - Show relationships between the system and external entities (people, other systems, organizations).
 - Reduce system costs and time by clarifying requirements early.
 - Include social and organizational factors affecting the system.

ATM System Context Diagram

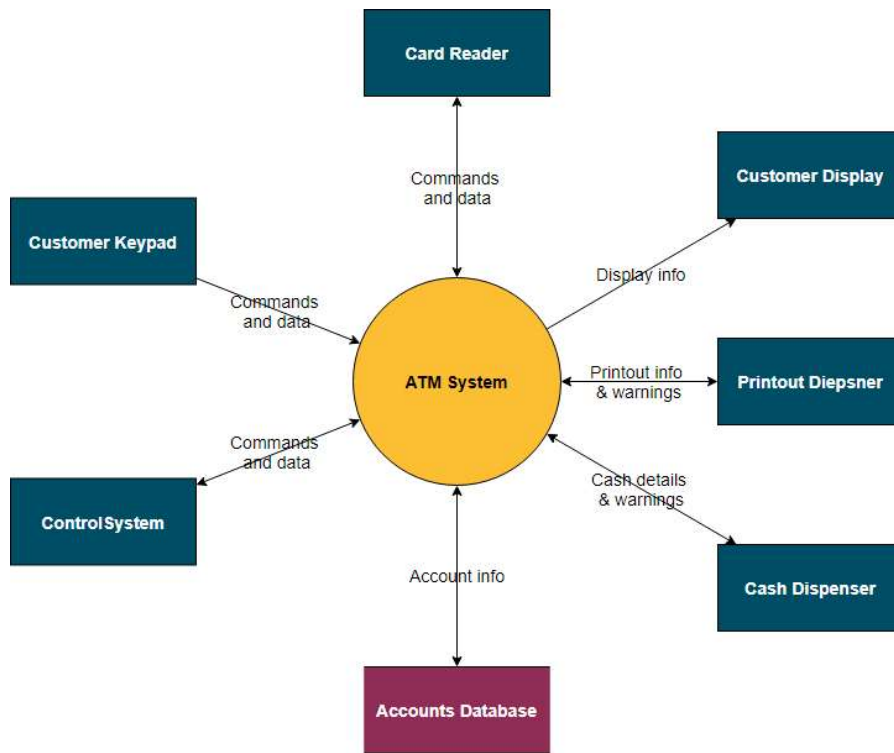
Shows the **ATM system and its interactions with external entities** (outside the system). It does **not show internal details**, just inputs, outputs, and interactions.

External Entities:

1. **Customer** – Interacts with ATM to withdraw cash, deposit money, check balance, or transfer funds.
2. **Bank Server** – Authorizes transactions, updates accounts, and sends transaction confirmations.
3. **Maintenance Personnel** – Replenishes cash, fixes hardware issues.

Inputs and Outputs:

- **From Customer to ATM:** Card details, PIN, transaction request.
- **From ATM to Customer:** Cash, receipt, balance information.
- **From ATM to Bank Server:** Transaction request, authentication request.
- **From Bank Server to ATM:** Transaction approval/rejection, updated balance.
- **From Maintenance Personnel to ATM:** Service requests, cash refill, maintenance logs



2. Interaction Models

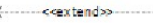
- **Purpose:** Show **how users and systems interact** and how different components of the system communicate.
- **Helps to:**
 - Identify user requirements.
 - Visualize how the system will be used.
 - Understand communication between system parts.

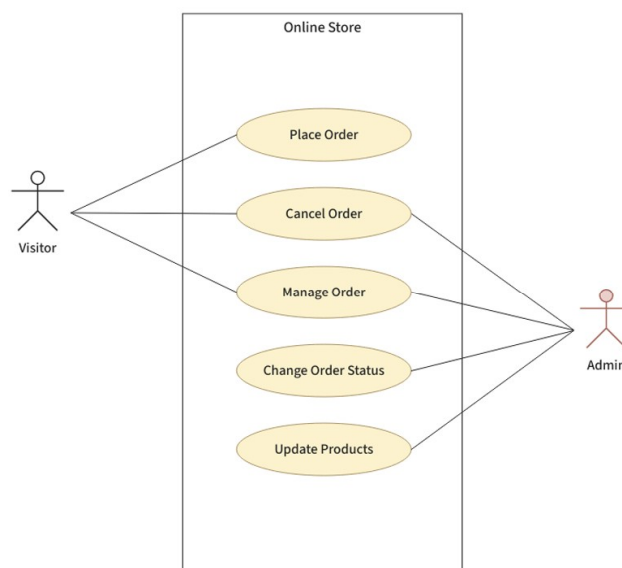
1) Use Case Modeling

- **Definition:** Describes **interactions between users (actors) and the system**.
- **Actor:** A human or another system interacting with the system.
- **Use Case:** A set of actions performed by the system to produce a result of value to an actor.
- **Example in Mental Health Care System:**
 - **Actors:** Patient, Therapist, Admin
 - **Use Cases:**
 - Patient books an appointment.
 - Therapist updates patient records.
 - Admin generates reports.

Use case diagrams visually show all actors and the actions they can perform.

NO	FIGURE	NAME	EXPLANATION
1		Actor	Functionality provided by the system as units that exchange messages between units or actors, is usually expressed using the verb at the beginning of the phrase name use case.
2		Use Case	People, processes, or other systems that interact with information systems that will be

NO	FIGURE	NAME	EXPLANATION
1		Actor	Functionality provided by the system as units that exchange messages between units or actors, is usually expressed using the verb at the beginning of the phrase name use case.
2		Use Case	People, processes, or other systems that interact with information systems that will be created outside the information system that will be created themselves.
3		Association	Communication between actors and use cases that participate in use cases <u>has interaction with</u> actors.
4		Include	Additional use case relations to a use case and use case that are added require this use case to carry out its functions.
5		Extend	Additional use case relations to a use case and use case that are added can stand alone even without additional use cases.

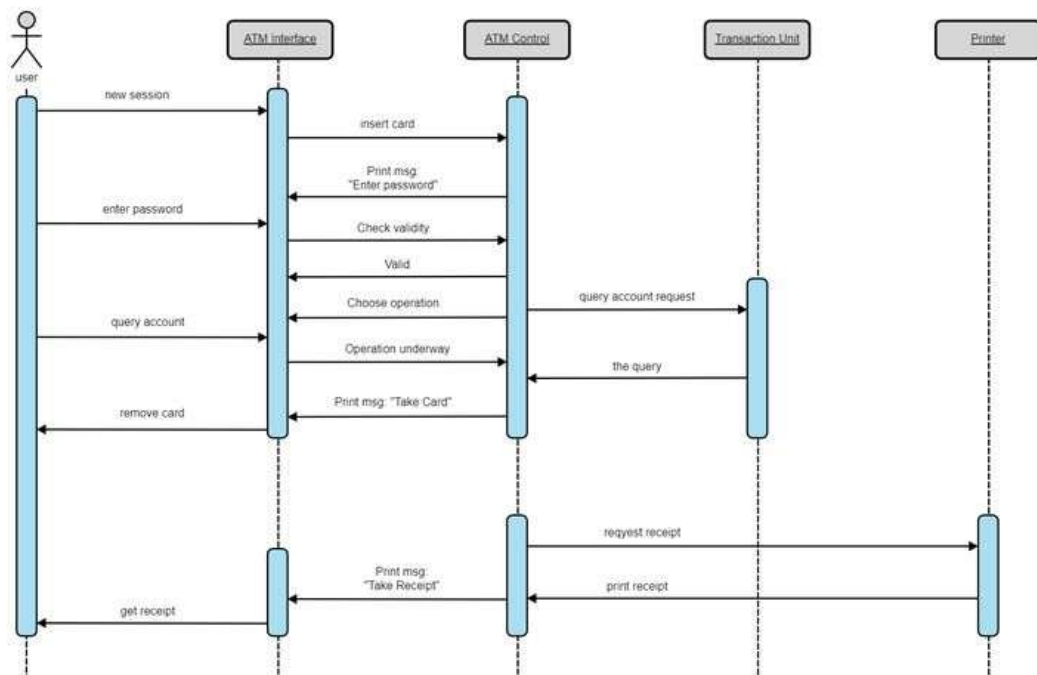


2. Sequence Diagrams (UML)

- **Definition:** Show the **sequence of interactions** between actors and system objects over time.

Key Entities (ATM Sequence Diagram)

- **Actor:** External entity interacting with the system
 - Example: Customer
- **Object:** Internal system component performing an action
 - Example: ATM Machine, Bank Server
- **Lifeline:** Existence of the object over time
 - Example: ATM Machine active while processing the transaction
- **Message:** Communication between objects
 - Example: Customer enters PIN → ATM validates with Bank Server
- **Activation:** Time during which an object performs an operation
 - Example: ATM checks account balance and dispenses cash



3. Structural Models

- **Purpose:** Show the **organization of the system** in terms of entities (objects, classes) and their relationships.
- Describe the **architecture or structure** of the system.

Class Diagram

A **class diagram** is a **static structural model** in UML that shows:

- **Classes** (blueprints of objects)
- **Attributes** (data or properties)
- **Methods/Operations** (functions or behaviors)
- **Relationships** between classes

It's mainly used in **object-oriented systems** to model the structure of the system.

Structure :

- Class:** Represented as a rectangle divided into three parts:
 - **Top:** Class name
 - **Middle:** Attributes (variables)
 - **Bottom:** Methods/operations (functions)
- Attributes:** Describe the data held by the class.
Example: balance: float
- Methods/Operations:** Actions the class can perform.
Example: withdraw(amount: float)
- Relationships:**

- **Association:** A link between classes (line connecting them)
- **Inheritance:** “is-a” relationship (arrow pointing to parent class)
- **Aggregation/Composition:** “has-a” relationship (diamond on the owner side)

Example: ATM System Class Diagram

Classes:

1) User

- ◆ Attributes: name, cardNumber, pin
- ◆ Methods: enterPin(), requestWithdrawal()

2) ATM

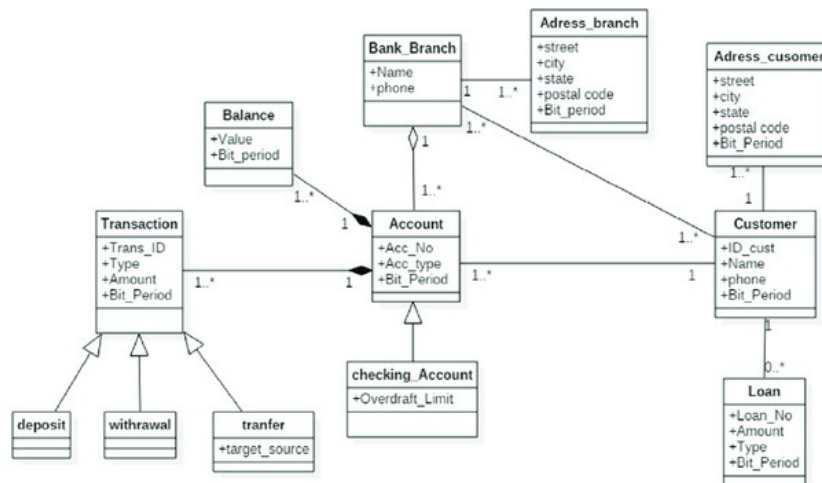
- ◆ Attributes: atmId, location
- ◆ Methods: readCard(), dispenseCash(), validatePin()

3) BankAccount

- ◆ Attributes: accountNumber, balance
- ◆ Methods: debit(amount), credit(amount)

Relationships:

- User “uses” ATM (Association)
- ATM “accesses” BankAccount (Association)



4. Behavioral Models

Behavioral models describe **how the system behaves dynamically** — that is, **how it reacts to inputs, events, or changes over time**.

They focus on **system behavior**, not structure, and are particularly useful for understanding the **workflow and logic** behind system operations.

These models help answer:

- What happens when a specific event occurs?
- How does data flow or change through the system?
- In what sequence are operations performed?

Types of Behavioral Models

1. Data-Driven Models

These models show **how data moves through the system**, how it's **transformed, processed, and stored** to produce outputs.

They focus on **data flow** rather than event triggers.

Used in:

- Financial systems
- Inventory management
- Data processing applications

Tools used:

- Data Flow Diagrams (DFD)
- Activity Diagrams

Example: Online Bookstore

Step	Process Description
Input:	Customer selects books and adds them to the cart.
Process:	System calculates the total cost, applies discounts, checks availability, and processes payment.
Output:	System generates an invoice and order confirmation.

Explanation:

Here, the focus is on **how data (book details, customer info, payment data)** flows through different processes — from input to output.

2. Event-Driven Models

Event-driven models show **how the system responds to specific events** (user actions or system triggers).

They focus on **control flow** — what happens **after** an event occurs.

Used in:

- Real-time systems
- Interactive systems (e.g., ATM, e-commerce, ticket booking)
- Systems where responses depend on external triggers

Tools used:

- State Machine Diagrams (Statecharts)
- Sequence Diagrams

Example: Online Bookstore

Event	System Response
Event: Customer clicks “Buy Now.”	System checks stock availability.
Event: Stock is available.	System confirms order and sends a confirmation message.
Event: Stock is unavailable.	System displays “Out of stock” message.

Explanation:

Here, behavior depends on the **events** triggered by the user and the **system’s reactions** to them. For example, clicking “Buy Now” could lead to different outcomes depending on conditions.

Model-Driven Architecture (MDA)

Model-Driven Architecture is an approach to **software design and implementation** that focuses on **developing and transforming models** instead of writing code directly.

It uses **UML (Unified Modeling Language)** models to describe a system at different levels of abstraction.

Three Levels of MDA Models:

1. Computation Independent Model (CIM):

- This model focuses on **what the system must do**, not **how** it will do it.
- It describes the **real-world environment** in which the system will operate — that is, the **business processes, entities, and goals**.
- It is independent of both **computation** (algorithms, data structures) and **implementation** (hardware, OS, language).
- Also called the **domain model**.
- Describes the **important domain concepts** used in the system — independent of computation or technology.
- Represents what the system is about (business or real-world view).

Usually represented using:

- **Use Case Diagrams** (to show business processes)
- **Class Diagrams** (to show domain entities)

- Example: Modeling hospital processes like “Patient Registration”, “Doctor Consultation”

2. Platform Independent Model (PIM):

- Describes **how the system operates** but without depending on any specific platform.
- Shows **system structure and behavior** using UML diagrams (class, sequence, activity diagrams, etc.).
- Example:

Continuing the **Hospital Management System**:

- The PIM might define:
 - A class Appointment with attributes date, time, and patientID.
 - A function scheduleAppointment() and its sequence of steps.
- Still doesn't say *whether it's in Java, C#, or SQL* — only the logic.

3. Platform Specific Model (PSM):

- A **transformation of the PIM** that includes details specific to a chosen platform.
- Each platform (e.g., Java, .NET) has its own PSM.
- Example: Implementing PIM logic using Java classes or SQL database schema.

Summary Table

Level	Full Form	Focus	Independent of	Example (Hospital System)
CIM	Computation Independent Model	Business/domain understanding	Software & hardware	Patient books appointment, doctor checks report
PIM	Platform Independent Model	System logic & behavior	Implementation platform	Class & sequence diagrams defining system logic
PSM	Platform Specific Model	Actual implementation	—	Java classes, MySQL schema, REST API design

Pros (Advantages):

- Systems can be designed at **higher levels of abstraction**, improving understanding.

Cons (Disadvantages):

- Models made for abstraction may **not fit perfectly for real implementation**.
- Developing translators (to convert models into code) for new platforms can be **time-consuming and costly**.

Unified Modeling Language (UML)

UML (Unified Modeling Language) is a **standard visual modeling language** used to **describe, design, and document** the structure and behavior of a software system.

It helps developers, analysts, and designers **communicate clearly** using **graphical diagrams**.

Purpose of UML:

1. To **visualize** a system before implementation.
2. To **specify** the structure and behavior of the software.
3. To **construct** a system model that can be transformed into code.
4. To **document** the design and architecture of the software.

Key Characteristics:

- **Graphical language** with standardized symbols.
- **Independent of programming language** or platform.
- Can model both **static (structural)** and **dynamic (behavioral)** aspects of a system.